Proc. of the International Conference on Electrical, Computer and Energy Technologies (ICECET)
20-22 July 2022, Prague-Czech Republic

2022 International Conference on Electrical, Computer and Energy Technologies (ICECET) | 978-1-6654-7087-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICECET55527.2022.9872665

Comparison of Choreography vs Orchestration Based Saga Patterns in Microservices

Sahin Aydin
Department of Information Technologies
Işık University
Istanbul, Türkiye
sahin.aydin@isikun.edu.tr

Cem Berke Çebi
Department of Computer Engineering
Işık University
Istanbul, Türkiye
20COMP5014@isik.edu.tr

***Abstract*— Microservice Architecture (MSA) is a design and architecture pattern created to deal with the challenges of conventional software programs in terms of stream processing, highly available flexibility, and infrastructural agility. Despite the many advantages of MSA, designing isolated services using the autonomous Databases per Services paradigm is difficult. We realized that because each microservice will have its repository, ensuring data coherence between databases becomes difficult, especially in reversals, where operations transcend different sites. Distributed networked transactions and rollbacks can be efficiently handled using two-phase commitment methods in hardware virtualization using RDBMS databases. However, these approaches can't be used in micro-services with segregated No-SQL servers. Three issues have been addressed in this study: (i) investigate the implementation of event choreography and orchestration methods for the Saga pattern execution in MSA, (ii) existing reality suggestions on the saga pattern adoption and implementation besides the use cases, and (iii) introduce the disbursed transaction records and rollbacks challenges in isolated No-SQL databases with reliant collections in MSA.**

Keywords— saga pattern; orchestration; microservice architecture; choreography; styling; SQL database

I. INTRODUCTION

Microservice architecture is a group of random applications, each of which serves a distinct purpose. These function components can't function properly in isolation, and that they need a way to communicate and share data. The microservice architectural design is a method of building a single application as a collection of discrete services. Each runs in its process and communicates through lightweight means, most commonly an HTTP resource API. Programmers have used two unique coordination methods to determine database management difficulties in distributed microservices: orchestration and choreography. Surprisingly, these two strategies have driven programmers to a point where they are probably amid monolithic authority and full in-dependence [1-2].

The majority of people believe that a microservice design is ideal for developing highly scalable. This isn't incorrect, but we should investigate the dimensions this architectural style scales best in. The first dimension that springs to mind are the quantity of load. This means that if the workload grows, it should be easy to add more re-sources to maintain the same efficiency level. This isn't the purpose of microservices in the first place. The centrally sponsored is the one that the microservice design scales. In many other words, adding new features and attributes to a brand at any point is simple. That leads to the conclusion that responding to impending load difficulties with new standards is simple. Observe a bunch of dancers performing on a stage to grasp better how these models work. When all of the dancers are in good sync, the dance appears appealing. Although each dancer may perform their moves, they work together to make the whole dance look pleasing. In many aspects, development environment changes the software development life cycle, techniques, and outcomes. Microservices were created as a result of monolithic architecture's inability to fulfill the demands of current applications. Although microservices solve several difficulties, the architecture also introduces a new set of issues. Ensuring system reliability be-tween microservices is one of those. The dancers could work together in several different ways. The backend for front ends pattern occurs when the orchestrator com-bines a succession of lower-level calls to assist in creating a website. The saga problem can be solved in a variety of ways, including orchestration. Without the use of a controller, services can connect. The asynchronous nature of choreography is one of the fundamental contrasts across orchestration and choreography. Then just sending a comment, delaying, and waiting for an answer, a service provider requests the events bus, which allocates work across the application's connected components. Each service is self-contained and performs its mission by consuming data related to it. Microservices design, an application development paradigm where an application or commercial use case is divided into a group of compostable services, promises corporate businesses a slew of performance aspects. For starters, they're compact, light, and simple to use. Second, they offer reusability, which lowers the cost of creating or up-dating applications, assures resource efficiency, and allows applications to extend on request [3-5].

At a high level, orchestration and choreography enable microservices to work together around a collective purpose. Because of its decentralized nature, choreography might be more complicated to comprehend than orchestration. Numerous services can pick up a message simultaneously, or it can move from service to broker to market to service. Without the need for a centralized orchestrator in charge of the transaction from beginning to conclusion, these interactions can be difficult to monitor. Further-more, the choreographic system is expensive to debug. In centralized data contexts, the saga system architecture is a method for maintaining system reliability across microservices. A saga design is a set of activities that refreshes each service and sends out a signal or event that triggers the next transaction phase. When a stage is broken, the saga pattern

978-1-6654-7087-2/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

conducts compensated transactions to put things right. Each saga pattern in microservices is responsible for its data throughout the data set more for every pattern. Nonetheless, this creates a fascinating situation. How happened when data from multiple saga-pattern microservices is used in a commercial transaction? Each saga pattern in microservices is responsible for its data throughout the information set more for every design. Nonetheless, this creates a fascinating situation. How happened when data from multiple saga-pattern microservices is used in a business arrangement? The saga pattern can be utilized to deal with this situation [6-8]. A Saga can be used to describe services that change the state. In a saga, every service that modifies the database's state in a networked transaction might trigger the next microservice by generating an incident. Inside the case of failure, the saga initiates a series of mitigating rollback events in the direction opposite from one service from the other. Considering the academic objectives of the group and the complexities of the surroundings while deciding among orchestration and choreography. Asynchronous programming is a difficult and novel way of thinking for certain groups; don't expect it easy to switch from a chronologically ordered to an asynchronous procedure.

Bigger businesses should choose a hybrid strategy, where more and more functions are choreographed, but those with significant interdependence are orchestrated via a controller. Autonomous operations will not fail if the resources they interact with are unavailable, but the natural interdependence will still be enforced.

Two strategies can be used to create these sagas: (i) event choreography, in which any service can initiate the events of other resources without the need for a centralized controller; (ii) orchestration, in which a central coordinator decides what events throughout the saga should be triggered. Both of these strategies have advantages and disadvantages depending on the use case. Some researchers have suggested use scenarios for which these methodologies are appropriate in the past, but no quantitative analysis has been done. This study attempted to suggest which saga methodology to employ in which scenario by analyzing performance and complexity using real-world data obtained by modeling a range of use cases with a unique project built on springs boot-driven microservices [9-10].

II. MOTIVATION

To cope with choreographies, many attempts and conventions have emerged. Even though choreographies have a more vital ability to scale as the number of com-plaints and resources increases, very little was done to properly and quantitatively il-lustrate this. A service-oriented architecture (SOA) is generated to help developers to create enormous application software that interacts via the Internet. Various services, primarily orchestrations and choreographies, can be created utilizing service composition techniques. Nevertheless, determining how the environment's features affect the composition's routing protocols is a challenging process.

Some scholars have already looked towards how microservice choreography and orchestration approaches are employed in microservice subsequent applications. Microservice simulation model raises additional issues, such as which service method is best for service collaboration. It's difficult to tell which method to creation is smarter: orchestration or choreography. The goal of this article is to examine different significant aspects in the orchestration and choreography procedure, such as data redundancy, resource usage, and memory requirements. In several simulation situations, the applicability of microservice orchestration and choreography is examined.

III. DATA MANAGEMENT PATTERNS FOR MICROSERVICES ARCHITECTURE

We run the danger of data inconsistency if we use numerous databases. Consistency is not addressed immediately when there are multiple data storage systems. This is something that needs to be taken into account when building the microservices architecture. Some actual data management strategies that can help you create da-ta-consistent microservices-based apps have been examined in the following sections.

Keep your text and graphic files separate until after the text has been formatted and styled. Do not use hard tabs, and limit use of hard returns to only one return at the end of a paragraph. Do not add any kind of pagination anywhere in the paper. Do not number text heads-the template will do that for you.

A. Saga Pattern

In centralized data contexts, the saga system architecture is a method for maintaining system reliability between microservices. A saga paradigm is a set of transactions that refreshes each service and sends out a communication or event that triggers the next transactions phase. When a stage is broken, the saga-pattern conducts compensate transactions to restore the balance. Every saga-pattern in microservices is responsible for its data in the information set per-service pattern. Nonetheless, this creates a fascinating situation. The saga pattern is a microservices architecture and it is for signing a consent form that spans multiple services is presented in Figure 1.

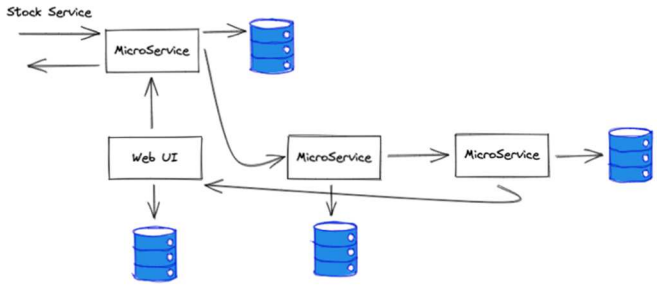


Fig. 1. Saga design pattern architecture

A sequence of local transactions makes up the saga structure. In a saga-pattern, each service completes its transaction and publishes an event. Various systems pick up on the event and carry out the next trade in the area. If a process fails for unexplained causes, the saga-pattern performs compensated operations to undo the prior transactions' effects [11].

A single request is spanning over multiple services, and need for atomicity of transaction and isolation of concurrent transactions. Transient state of request process between services and roll back in case of failure. To maintain the data consistency saga design pattern is used, so moving further we look at the basics of saga different pattern. In saga pattern, each transaction

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

to a particular service is treated as local transaction. So, we have sequences of local transactions. Each local transaction up-dates data within a single service and transactions could be sequential or parallel. This design pattern preferred for long running transactions.

Mainly, two types of implementation of saga pattern is mostly used: event-based approach and orchestration-based approach.

1) Choreography-Based Saga

There is no central orchestrator in this system. Each service in the saga completes its transaction and distributes events. Following up on specific circumstances and conducting their operations are handled by various services. Similarly, they are not permitted to disperse separate occasions events based on the scenario. The Saga Paradigm is a popular microservices Data Management pattern for applications to per-form many transactions. It consists of a series of local operations, each of which produces an event describing the status of the inquiries that have been initiated. Since the other services are dependent on the quality of the prior services, the saga will immediately undo any subsequent transactions that have repeatedly failed; whenever consumer purchases at an e-commerce store, the client service and order service departments are both active. Whenever a customer support representative sends an order, it will be marked as waiting [12].

The saga would handle the event placement by contacting the e-commerce web-site via the order service. When the order service receives verification of the order, it responds. Microservices are also built with vertical high scalability. Microservices are conscience, independent, and decoupled. Dependency-imposed constraints are neatly removed, allowing failures to be accepted while achieving the essential isolation. Microservices software developers can execute on business needs more quickly and autonomously. However, there are several exceptions.

Microservices-centric systems have introduced some new operational issues. Microservices should be discovered dynamically. Management and data flows must be accurately channeled to the proper and operational microservices after determining the location based addresses as shown in Figure 2.

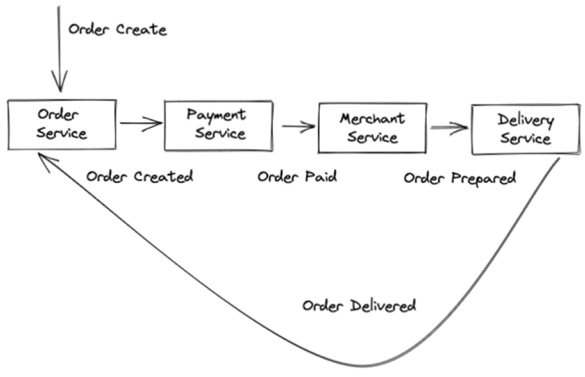


Fig. 2. Choreography event-based architecture

Choreography event-based architecture is suitable for simple labor procedures with a small number of participants and no requirement for a coordinating rationale. While adding more strides, the work procedure can become perplexing because it's challenging to keep track of its saga members responding to and which directions. It is simple to put into practice, easier to manage and maintain the state, procedures, and monitoring.

The saga will accept or disapprove the directive based on the response. First, the consumer is given a proper order status, either that it will be supplied or that the buyer should continue to the payment option. If the e-commerce store does not accept the order, it will send an apologetic message or display some out the sign [13-14].

2) Orchestration-Based Saga

An orchestration-based saga in this approach interacts with each activity and encourages the service registry to do neighboring transactions depending on occurrences [15]. This orchestrator can also be thought of as a saga supervisor as shown in Figure 3.

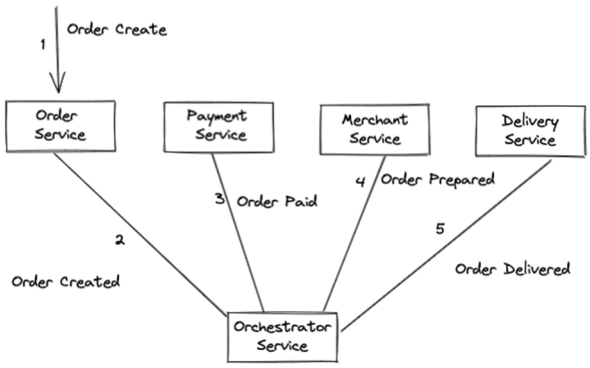


Fig. 3. Architecture of the orchestration based saga

Orchestration based saga is useful for complex work processes, including numerous members or new members added over the long run. Additional system design necessitates the use of coordination logic.

B. Advantages and Disadvantages of Saga Pattern

The saga pattern's main benefit is that it maintains system reliability throughout multiple services without tight connection. The saga patterns microservices rely heav-ily on this. Regardless, the saga pattern's biggest downside is its seeming complexity from a computing standpoint. Furthermore, developers are unfamiliar with coding sagas as making this determination. The other criterion is that compensatory activities must be explicitly designed to make sagas operate. The saga structure spring boot is an application framework that allows you to cope with deep and expensive ACID transactions in a flexible fashion. It takes a financial transaction and breaks it down into individual advances within each activity. Revisiting past performance provides overall stability. Orchestration and choreography are two different methods of micro-services operation, and one may be preferable for certain business activities. Orchestration is more effective for transactional services or services that rely on others to per-form a task.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I. MICROSERVICE ORCHESTRATION VS. CHOREOGRAPHY

Orchestration	Choreography
In an orchestra, the conductor is known as the "orchestrator." As a result, he is in charge of evoking all of the sounds.	The alternative method for achieving microservice interaction is through choreography. We wish to avoid dependency in a microservice architecture.
Likewise, the orchestrator (central controller) manages all microservice connections in microservice orchestration. It both communicates and replies to information.	Choreography overcomes the problem that was the most challenging part of the orchestration process.
The application is a microservice. It is like orchestration is a centralized service. It makes a call to one provider and awaits an answer while moving on to another.	Choreography is a decentralized method of disseminating data known as events. Services that are interested in certain events use it and take action.
Because we centralize the business requirements, it's easy to maintain and administer.	This asynchronous solution eliminates the waiting time that occurs in orchestration due to request/response processing.
The application's flow is efficiently coordinated in synchronous processing.	Processing is faster since there is no reliance on a central controller.
Due to connected services, dependency is created. If service A is unavailable, for illustration, service B will never be contacted.	It's simple to add and change information. In addition, the services can be withdrawn or added to the event stream at any moment.
The orchestrator is the solitary person in charge. The execution stops, and the program fails if it starts going down.	There is no single point of failure because the control is spread.
The orchestrator is in charge of everything. The processing halts, and the program fails if it goes down.	Teams focus on specific services rather than the complete program, which fits nicely with the agile delivery approach.
Because we don't track business processes, we lose visibility.	There are a variety of patterns that can be employed. Event source, for example, is a method of storing and replaying events. In addition, read and write activities can be separated using command query responsibility segregation (CQRS).
Orchestration is based on a centralized service known as the orchestrator or flow chart processor, which serves as the "brains" for asking services to execute specified activities and anticipating a message back to the overall understanding, making judgments about the next plan of events.	The issue of complexity is a source of concern. Each service is self-contained to determine the logic and react to the data in the event stream.
Because all of the logic is contained in a single central area, the business method is straightforward to administer and modify.	Because the business process is dispersed, it is difficult to maintain and oversee the whole process.
Callable services will also be able to conduct action and reply without the need for pertaining.	In the asynchronous approach, a shift in mindset is required.

On the other hand, Choreography is well suited to work that is asynchronous, self-contained, and decoupled. Take, for example, the front end of a web service, such as a website for an e-commerce company. Users may use that front end to perform tasks such as search, add to cart, and checkout. When a customer adds an item to their shopping, they expect a response. The cart will seem empty if the application has not yet picked up that selection message and added the modification to the database. Even if the delay is only 30 seconds, it can generate confusion, and the client may decide to add the item again. There are now two products in the cart after the services have completed their interactions. When users encounter these problems, their experience suffers [16].

C. Using Saga Pattern to Maintain Data Consistency

Sagas are technologies that allow a microservice architecture to guarantee data consistency without the usage of distributed transactions. For each system command that needs to update data in various services, you create a saga. A saga is a series of interconnected local interactions. Each local transaction uses the well-known ACID transaction frameworks and libraries to update data within a single service. When SAGA is finished, send a response. The conclusion of the SAGA is included in the answer using this manner. This is essentially a waiting call, and that it can result in a reduction in business continuity. Send your feedback as soon as possible. The out-come of the saga is not included in the service response in this way.

The client must poll or be alerted of the saga's development. To alert the client, an occasion approach might be used. The saga design enables a program to ensure system reliability throughout many processes while relying on deciding how much money (two-phase commits) and avoiding the issues outlined in the preceding article. On the other hand, the programming paradigm is extremely complex: for instance, each total transaction

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

developer must create a compensatory transaction that restores the modifications performed previously in the saga. One of the primary issues in microservices and other decentralized innovation advantages is data integrity across many services.

With the ACID features of database transactions, it was not a problem for a monolithic design style. One method to solve this problem in decentralized patterns is to use SAGS.

That's where the Saga pattern shines, as it assists in [17-19]:

- Preserve data consistency avoiding tight connection across various micro-services.
- When compared to 2PC, it perform better
- There should be no one reason for failure.
- Ultimately, maintain the total transaction overall state consistent.

IV. Results and Discussion

In the context of SOA, choreography and orchestration refer to the utilization of processes spanning several participants, with message traffic traveling in all directions according to a complicated set of rules. Choreography and orchestration are attempts to bring all of this activity under control or to coordinate it. They approach the problem by putting rigor into how message exchanges are represented and arranging the whole process with the appropriate control-flow patterns. Inter-organizational use cases (for example, B2B commerce involving a customer, seller, and wholesaler) or intra-organizational use cases (if the company is large enough and the participants function as independent entities) are both possible [20]. It's complicated and tough to maintain data consistency in a microservices environment when each microservice has its database system. Saga is one of the finest ways to implement BASE transactions in a centralized network and assure data consistency. Saga's successful adoption necessitates a shift in thinking on the part of both the business operational teams. The saga design enables a program to preserve data consistency across many processes without relying on deciding how much money (two-phase commits) and avoiding the issues outlined in the preceding article. On the other hand, the programming model is highly complex: for instance, each transaction's programmer must create a compensatory transaction that eliminates the modifications performed previously in the saga. Programs must collaborate in real-world use scenarios. Both are required: Depending on context, a choreographed incident approach or an orchestrated function approach can be used. When employing microservices, you can get the kind of separation you want with a creative mix. A cloud-based application is frequently broken into numerous small services that operate together to perform a business arrangement end-to-end in monolithic application. Every service is suitable for a single business process to reduce coupling between them. Important and strongest, a lower code base, and scalability are just a few of the advantages. Designing an efficient and scalable process, on the other hand, is difficult and frequently necessitates complicated organization chart interaction. If you plan to upgrade, delete, or add additional services often, use the choreography design. With little work and no damage to existing systems, the whole software can be updated. If you're having trouble with bottlenecks in the central orchestrator, think about this approach.

This pattern is a perfect fit for a serverless architecture in which all services are temporary or occasion. Service can start up in response to an event, perform their work, and then be deactivated once the work is completed.

V. Conclusion

We detailed how we used orchestration and choreography to implement dynamic in this paper. This project aims to compare and contrast the two methods to orchestration and choreography. The analysis is based on the amount of time, memory, and power used. We are able to clearly establish that event choreography is substantially faster in performance using the study based on time consumption and memory. When several events are triggered from each micro service, however, event choreography be-comes extremely difficult to create and manage. It's also clear that managing many trigger actions without a central orchestrator is difficult, because one developer or team working on a service may not be aware of the other's efforts. This demonstrates that choreography-based composition is a recommended method when there are fewer microservices involved in the distributed business process or when the number of event triggers is not too complicated. Orchestration is sluggish, but it comes in handy when dealing with complex transaction scenarios. Services must collaborate in re-al-world use scenarios. Based on situation, you'll need the: a choreographed occasion approach and an orchestrated function one. By employing microservices, you can get the level of independence you want with a creative mix.

References

[1] S. Neha, U. Sakthivel, and P. Raj. “Selection Mechanism of Micro-Services Orchestration vs. Choreography.” International journal of Web & Semantic Technology 10.1 (2019): 01–13.

[2] S. Andreas et al. “Orchestration vs. Choreography Functional Association for Future Automation Systems.” IFAC-PapersOnLine 53.2 (2020): 8268–8275.

[3] B. Fadhlallah, N. Le Sommer, and Y. Maheo. “Choreography-Based vs Orchestration-Based Service Composition in Opportunistic Networks.” International Conference on Wireless and Mobile Computing, Networking and Communications. Vol. 2017-October. IEEE Computer Society, 2017.

[4] S. Martin, O. Chaloupka, and B. Rossi. “The Saga Pattern in a Reactive Microservices Environment.” ICSOFT 2019 - Proceedings of the 14th International Conference on Software Technologies. SciTePress, 2019. 483–490.

[5] R. Nicole, “Title of paper with only first word capitalized,” J. Name Stand. Abbrev., in press.

[6] D. Karolin, R. Lichtenthäler, and G. Wirtz. “An Evaluation of Saga Pattern Implementation Technologies.” CEUR Workshop Proceedings. Vol. 2839. CEUR-WS, 2021. 74–82.

[7] D. Nicola et al. “Microservices: Yesterday, Today, and Tomorrow.” Present and Ulterior Software Engineering. Springer International Publishing, 2017. 195–216.

[8] G.-M. Hector, and K. Salem. “Sagas.” ACM SIGMOD Record 16.3 (1987): 249–259.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.

[9] X. Gang et al. “Reaching Consensus in Decentralized Coordination of Distributed Microservices.” Computer Networks 187 (2021): n. pag. Computer Networks. Web.

[10] D. Fei et al. “A Choreography Analysis Approach for Microservice Composition in Cyber-Physical-Social Systems.” IEEE Access 8 (2020): 53215–53222. IEEE Access.

[11] N. Adriatik, Mathias Weske, and Jan Mendling. “Semi-Automatic Derivation of RESTful Choreographies from Business Process Choreographies.” Software and Systems Modeling 18.2 (2019): 1195–1208.

[12] V. Pedro, V. Torres, and V. Pelechano. “A Microservice Composition Approach Based on the Choreography of BPMN Fragments.” Information and Software Technology 127 (2020)

[13] W. Andreas et al. “Model-as-You-Go for Choreographies: Rewinding and Repeating Scientific Choreographies.” IEEE Transactions on Services Computing 13.5 (2020): 901–914.

[14] J. Dube, Tafadzwa, and G. İnce. “A Novel Interface for Generating Choreography Based on Augmented Reality.” International Journal of Human Computer Studies 132 (2019): 12–24.

[15] Grange, Ray, Graham Heaslip, and Caroline McMullan. “Coordination to Choreography: The Evolution of Humanitarian Supply Chains.” Journal of Humanitarian Logistics and Supply Chain Management 10.1 (2020): 21–44.

[16] L. Junio C., Ricardo C.A. Rocha, and Fabio M. Costa. “An Approach for QoS-Aware Selection of Shared Services for Multiple Service Choreographies.” Proceedings - 2016 IEEE Symposium on Service-Oriented System Engineering, SOSE 2016. Institute of Electrical and Electronics Engineers Inc., 2016. 129–138.

[17] R. Charlotte, Annouk Lievens, and Vera Blazevic. “Hybrid Orchestration in Multi-Stakeholder Innovation Networks: Practices of Mobilizing Multiple, Diverse Stakeholders across Organizational Boundaries.” Organization Studies 42.1 (2021): 61–83

[18] P. Shan L., Miao Cui, and Jinfang Qian. “Information Resource Orchestration during the COVID-19 Pandemic: A Study of Community Lockdowns in China.” International Journal of Information Management 54 (2020)

[19] S. de Sousa, Nathan F. et al. “Network Service Orchestration: A Survey.” Computer Communications 15 June 2019: 69–94.

[20] G. Anushri, P. Panagiotopoulos, and F. Bowen. “An Orchestration Approach to Smart City Data Ecosystems.” Technological Forecasting and Social Change 153 (2020)

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:46 UTC from IEEE Xplore. Restrictions apply.